Smart Commissioning Tool — Functional & Technical Specification

1. Purpose of this Specification

This specification defines the required functionality, behaviour, user interface expectations, data handling logic, and acceptance criteria for the **Smart Commissioning Tool**.

The document is intended for use by a software engineer, development team, or AI software agent to design and build the application based on the accompanying React UI mockup.

The tool is intended to support smart building commissioning activities, specifically around:

- Network device discovery
- BACnet device and point discovery
- MQTT topic, asset and payload discovery
- Data validation against expected project registers
- BACnet-to-MQTT translation validation
- Commissioning evidence reporting

The tool should be developed as a practical commissioning and validation platform for Master Systems Integrator activities, allowing engineers to confirm that systems and data are visible, structured, complete, accurate, and suitable for integration into a Smart Building environment.

---

2. Product Name

The product shall be named:

Smart Commissioning Tool

The name shall be used consistently across:

- Application header
- Browser/page title where applicable
- Reports
- Export files
- Generated evidence packs
- Login/sign-in screens if developed later
- Help and guided tour content

---

3. High-Level Requirement

The Smart Commissioning Tool shall provide a web-based interface that allows a commissioning engineer to configure system connectivity, discover devices and data sources, inspect live data,

compare discovered data against expected registers, validate BACnet and MQTT data quality, and generate evidence reports.

The core pages to be built are:

1. Configuration
2. IP Scanner
3. BACnet Discovery
4. MQTT Discovery
5. Data Validation
6. Reports

The application shall follow the visual layout, navigation structure, and interaction model shown in the accompanying React UI mockup.

---

4. Why the Tool is Needed

Smart building commissioning typically requires confirmation that devices are correctly installed, visible on the network, configured with the correct communication parameters, exposing the correct protocol data, and transmitting usable data to the Smart environment.

Manual validation using separate tools such as IP scanners, BACnet explorers, MQTT clients, spreadsheets and ad hoc reports is time-consuming, inconsistent, and difficult to evidence.

The Smart Commissioning Tool is required to provide a structured workflow that allows commissioning teams to:

- Confirm expected devices are visible on the network.
- Identify unexpected or rogue devices.
- Confirm BACnet devices are discoverable and exposing the required objects.
- Confirm MQTT topics are subscribable and payloads are correctly structured.
- Validate that expected assets and points are present.
- Validate live values, units, timestamps, status flags and quality indicators.
- Compare BACnet source data against MQTT translated data.
- Produce issue registers and commissioning reports.
- Provide clear evidence of system readiness for MSI and Smart Building integration.

---

5. Where the Tool Applies

The tool applies to projects where IP-connected, BACnet-connected, MQTT-connected or translated Smart Building data must be commissioned and validated.

Typical systems include, but are not limited to:

- Building Management Systems
- Energy Metering Systems
- Lighting Control Systems
- Access Control Systems

- CCTV and Security Systems
- Fire monitoring interfaces where applicable
- Lift and vertical transportation interfaces
- IAQ sensors and IoT gateways
- Occupancy and people-counting systems
- Smart lockers and other tenant/amenity systems
- MQTT brokers and Smart Building data platforms

The tool should be suitable for use during:

- Factory acceptance testing
- Site acceptance testing
- Network readiness testing
- System integration testing
- Smart Building data validation
- Commissioning evidence production
- Defects and issue resolution
- Handover validation

---

6. User Roles

6.1 MSI / Smart Commissioning Engineer

Primary user of the application.

Expected activities:

- Configure scan and protocol settings.
- Import project registers.
- Run discovery workflows.
- Review discovered devices and points.
- Inspect live data.
- Run validation checks.
- Raise or export issue registers.
- Produce commissioning evidence.

6.2 Software / Platform Engineer

Uses the output of the tool to troubleshoot protocol, payload or integration issues.

Expected activities:

- Review MQTT payloads.
- Review BACnet-to-MQTT mapping results.
- Investigate schema issues.
- Review missing, stale or out-of-tolerance data.

6.3 Project Manager / Delivery Lead

Uses reports and summary views to understand commissioning progress.

Expected activities:

- Review high-level device and data readiness metrics.
- Track issue counts.
- Confirm evidence production status.

6.4 Backend / System Administrator

Configures deployment, access, services, certificates and storage.

Expected activities:

- Maintain broker credentials.
- Maintain certificates and keys.
- Configure backup and logging.
- Support network and protocol connectivity.

---

7. Overall Application Behaviour

The application shall behave as a guided commissioning workflow.

The general workflow shall be:

1. Configure network, BACnet, MQTT, certificate, time and backup settings.
2. Import expected registers.
3. Perform IP discovery.
4. Review visible, missing and rogue IP devices.
5. Perform BACnet discovery.
6. Review BACnet devices and live object data.
7. Perform MQTT discovery.
8. Review MQTT topics, payloads and extracted points.
9. Import validation mapping files.
10. Validate BACnet data.
11. Validate MQTT payload data.
12. Compare BACnet source data to MQTT translated data.
13. Review device and point-level issues.
14. Export reports and issue registers.

The tool should not only show pass/fail results. It should allow the user to click into the underlying device, point, topic, object or mapping record to understand the reason for the result.

---

8. User Interface Requirements

8.1 General UI Style

The application shall follow the style shown in the React UI mockup:

- Light grey application background.

- White content cards.
- Rounded corners.
- Subtle shadows.
- Blue primary accent colour.
- Green status indicators for healthy/pass/online states.
- Amber status indicators for partial/warning states.
- Red status indicators for fail/error/unexpected states.
- Pill-style buttons and status tags.
- Compact but readable data tables.
- Grouped top navigation.
- Clickable rows in tables.
- Right-hand inspector/detail panels.
- Contextual information icons.

8.2 Header

The top header shall include:

- ELECTRACOM logo area.
- Product title: Smart Commissioning Tool.
- Version indicator.
- Back button.
- Home button.
- Export button.
- File menu.
- Secure/status pill.
- Brief button.
- Guided Tour button.
- Sign-in/user control area.
- User initials/avatar.

8.3 Secondary Navigation

The application shall use grouped navigation:

- SETUP
- Configuration
- DISCOVERY
- IP Scanner
- BACnet Discovery
- MQTT Discovery
- VALIDATION
- Data Validation
- WORKFLOW
- Reports

The active page shall be clearly highlighted with a blue pill/button state.

8.4 Breadcrumbs

Each page shall include a breadcrumb in the format:

`Home > Page Name`

Example:

`Home > BACnet Discovery`

8.5 Information Icons

Information icons shall be provided beside key labels, headings and metrics.

When clicked, the information icon shall display a popover explaining:

- What the element means.
- Why it is relevant.
- How the value is used.
- What the user should do if the value indicates a problem.

Information icons shall be clickable, not purely decorative.

They shall be provided for, at minimum:

- Configuration page banner.
- IP target range.
- Scan mode.
- Ports/services.
- Import register.
- Expected devices.
- Reachable devices.
- Rogue hosts.
- BACnet network number.
- BBMD settings.
- Object count.
- BACnet present value.
- BACnet reliability.
- MQTT broker status.
- MQTT root topic filter.
- MQTT QoS.
- Payload status.
- Extracted points.
- Mapping comparison.
- Point issues.

---

9. Configuration Page Specification

9.1 What Needs to be Built

Build a Configuration page that allows the commissioning engineer to configure all application-level and protocol-level settings required by the tool.

The page shall include configuration sections for:

- Device IP settings
- BACnet discovery settings
- MQTT broker and subscription settings
- Certificates and keys
- Time and NTP
- Backups and restore
- Logging and diagnostics

9.2 Why it is Needed

Discovery and validation workflows depend on correct network settings, protocol settings, broker connectivity, certificates, accurate time and reliable logs.

Without correct configuration, the tool may fail to discover devices, fail to subscribe to MQTT topics, incorrectly timestamp validation results, or be unable to generate reliable evidence.

9.3 Where it Applies

This page applies globally to the application instance or gateway service running the Smart Commissioning Tool.

The settings may apply to:

- Local gateway configuration.
- Discovery service configuration.
- MQTT client configuration.
- BACnet discovery engine configuration.
- Logging and diagnostics service.
- Backup service.

9.4 Required Fields

Device IP Settings

Required fields:

- Hostname
- IP assignment mode: DHCP or static IP
- IP address
- Subnet mask
- Gateway
- DNS servers
- VLAN ID, optional
- Network status

BACnet Settings for Discovery

Required fields:

- BACnet network number
- UDP port, default 47808
- Device instance range
- BBMD enabled/disabled
- BBMD address
- BBMD UDP port
- Foreign device enabled/disabled
- TTL value
- BACnet listener status

MQTT Broker & Subscription Settings

Required fields:

- MQTT broker FQDN or IP address
- Port, typically 8883 for MQTT over TLS
- Client ID
- Root topic
- QoS setting
- Keep alive interval
- Broker connection status

Certificates & Keys

Required fields:

- CA certificate
- Client certificate
- Private key
- Optional private key password
- Certificate validity status
- Certificate expiry date where available

Time & NTP

Required fields:

- Timezone
- Primary NTP server
- Secondary NTP server, optional
- NTP update interval
- Last sync time
- NTP status

Backups & Restore

Required fields:

- Backup schedule
- Backup retention
- Encrypted backup enabled/disabled
- Backup location
- Last backup status
- Restore action

Logging & Diagnostics

Required fields:

- Log level
- Log retention
- Remote syslog target, optional
- Syslog port
- Diagnostics mode
- Disk usage
- Logging status

9.5 How it Should Behave

The page shall allow the user to:

- View existing configuration.
- Edit configuration fields.
- Save configuration.
- Import configuration.
- Validate field formats.
- View status of network, BACnet listener, MQTT broker, certificates, NTP, backups and logging.
- Open information popovers for each major setting.

The page shall prevent invalid configuration from being saved where possible.

Examples of validation:

- IP address must be a valid IPv4 or IPv6 address.
- BACnet UDP port must be numeric.
- BACnet network number must be within valid range.
- MQTT port must be numeric.
- Root topic must not be empty.
- Certificate files must use supported file formats.
- NTP interval must be within permitted range.

9.6 Finished Result

The finished page shall look like the Configuration page in the React mockup:

- Multiple white configuration cards.

- Each card has a title, icon and information icon.
- Fields are clearly grouped.
- Status chips show current health.
- Import and Save buttons are available at the bottom.

9.7 Success Checks

Success shall be checked by confirming that:

- All required sections are present.
- All required fields can be displayed.
- Editable fields can be changed and saved.
- Invalid values are rejected or clearly flagged.
- MQTT broker connection status can be shown.
- BACnet listener status can be shown.
- Certificate validity can be shown.
- NTP sync status can be shown.
- Information icons open meaningful help popovers.

---

10. IP Scanner Page Specification

10.1 What Needs to be Built

Build an IP Scanner page that allows users to scan a defined IP range, identify visible devices, detect open ports/services, compare results against an imported IP Device Register, and inspect discovered devices.

The page shall include:

- Scan control panel.
- Register import function.
- Summary metric cards.
- Discovery results table.
- Clickable table rows.
- Right-hand device inspector.
- Live data and diagnostics tabs.

10.2 Why it is Needed

The IP Scanner is required to confirm that expected network-connected devices are visible on the CNS or project network.

It is also required to identify:

- Missing expected devices.
- Unexpected or rogue hosts.
- Incorrect IP addressing.
- Unexpected open services.
- Devices exposing unencrypted or unapproved services.
- Differences between the expected IP Device Register and the observed network state.

10.3 Where it Applies

The IP Scanner applies to all IP-connected systems and devices included within the commissioning scope.

Typical device types include:

- BMS controllers
- Metering gateways
- Lighting gateways
- Access control panels
- CCTV devices
- IoT gateways
- MQTT brokers
- Virtual machines
- Head-end servers
- Network-connected plant controllers

10.4 Required Inputs

The page shall support the following inputs:

- Target IP range or CIDR subnet.
- Scan mode.
- Ports/services to scan.
- Timeout setting.
- Optional advanced scan settings.
- Imported IP Device Register.

10.5 IP Device Register Import

The application shall allow an IP Device Register to be imported.

Minimum expected columns:

- Project/site
- System
- Device name
- Asset ID, if available
- Expected hostname
- Expected IP address
- Subnet/VLAN
- Device type
- Manufacturer/vendor
- Expected services/ports
- Location
- Notes

The tool shall compare discovered devices against the imported register.

10.6 Required Outputs

The page shall show:

- Expected devices count.
- Reachable devices count.
- Register matches count.
- Unexpected/rogue hosts count.
- Discovery results table.
- Device-level inspector panel.
- Exportable results.

10.7 Discovery Results Table

The table shall include, at minimum:

- IP address
- Hostname
- Status
- Latency
- Open ports
- Detected service
- Register status
- Project/site

Rows shall be clickable.

Clicking a row shall open or update the device inspector panel.

10.8 Device Inspector

The right-hand inspector shall show details for the selected device.

Required sections:

- Device overview
- Live health
- Network details
- Observed services
- Register comparison
- Notes

Required tabs:

- Overview
- Live Data
- Diagnostics

Overview Tab

The Overview tab shall show:

- Hostname
- Device type
- Vendor
- Model
- Description
- IP address
- Subnet mask
- Gateway
- DNS server
- VLAN ID
- Project/site
- Location
- Observed services
- Register comparison summary

Live Data Tab

The Live Data tab shall show live or latest observed values such as:

- Response time
- Packet loss
- Link status
- MAC address
- Last seen timestamp
- Active sessions where available
- Service uptime where available
- Protocol response health where available

Diagnostics Tab

The Diagnostics tab shall show:

- Detected warnings.
- Unexpected open ports.
- Missing expected services.
- Register comparison issues.
- Suggested actions.

10.9 Status Logic

Suggested status values:

- Reachable
- Unreachable
- Rogue Host
- Match
- Partial
- Not in Register

- No Data

Register matching logic should consider:

- IP address match.
- Hostname match.
- Expected port/service match.
- Device type match where available.
- Project/site match.

10.10 How it Should Behave

The user shall be able to:

- Enter an IP range.
- Select scan mode.
- Select ports/services.
- Import an IP Device Register.
- Start a scan.
- View progress where scans are active.
- View summary metrics.
- Sort and filter results.
- Click a discovered device.
- View live device details.
- Add notes.
- Export results.
- Compare results to register.
- Open device history.

10.11 Finished Result

The finished page shall look like the IP Scanner page in the React mockup:

- Top control cards.
- Four summary metric cards.
- Large results table on the left.
- Device inspector panel on the right.
- Clickable information icons.
- Clickable discovery rows.

10.12 Success Checks

Success shall be checked by confirming that:

- A user can run a scan against a defined IP range.
- Discovered devices are listed in the table.
- Imported register records are compared against discovered results.
- Expected, reachable, matched and rogue metrics are calculated.
- Clicking a row updates the device inspector.
- The device inspector shows relevant live data.
- Results can be exported.
- Information icons are clickable and explain the relevant UI elements.

11. BACnet Discovery Page Specification

11.1 What Needs to be Built

Build a BACnet Discovery page that allows users to discover BACnet devices, inspect BACnet properties and objects, compare results against a BACnet Device Register, and view live BACnet point values.

The page shall include:

- BACnet discovery controls.
- BACnet register import.
- Summary metric cards.
- BACnet device results table.
- Clickable BACnet device rows.
- Device inspector panel.
- Live BACnet points table.
- Object tree view.

11.2 Why it is Needed

BACnet discovery is required to confirm that BACnet devices are visible, correctly configured and exposing the required objects for integration and validation.

It is also required to confirm:

- BACnet/IP routing is working.
- BBMD or foreign device configuration is correct where used.
- Expected BACnet devices are online.
- Device instances are correct.
- Object lists can be read.
- Present values are available.
- Status flags and reliability can be checked.
- Required points are present before translation or Smart integration.

11.3 Where it Applies

This page applies to all BACnet devices within the commissioning scope.

Typical devices include:

- BMS controllers
- Plant controllers
- AHU controllers
- VAV controllers
- FCU controllers
- Metering gateways exposing BACnet
- Lighting BACnet gateways
- Fire or lift monitoring gateways where applicable

11.4 Required Inputs

The page shall support:

- BACnet network number.
- UDP port.
- Device instance range.
- BBMD settings.
- Foreign device settings.
- TTL setting.
- BACnet Device Register import.

11.5 BACnet Device Register Import

Minimum expected columns:

- Project/site
- System
- Device name
- BACnet device instance
- BACnet network number
- IP address
- UDP port
- Vendor/manufacturer
- Device type
- Expected object count, if available
- Location
- Notes

Optional columns:

- Expected object IDs
- Expected object names
- Expected units
- Expected COV settings
- Expected polling intervals

11.6 Required Outputs

The page shall show:

- Expected devices.
- Discovered devices.
- Objects discovered.
- Unmatched/unexpected devices.
- BACnet device table.
- BACnet object data.
- Object tree.
- Register comparison results.

11.7 Discovery Results Table

The table shall include:

- Device instance
- Device name
- IP address
- BACnet network
- UDP port
- Object count
- Register status
- Online status

Rows shall be clickable.

Clicking a row shall update the BACnet device inspector.

11.8 BACnet Device Inspector

The BACnet inspector shall include:

- Device title
- Online/matched badges
- Overview tab
- Live Points tab
- Object Tree tab
- Action buttons

Required action buttons:

- Read Properties
- Export Objects
- Compare to Register

11.9 Overview Tab

The Overview tab shall show:

- Device name
- Device instance
- Vendor
- Model
- Firmware
- Location
- BACnet network number
- IP address
- UDP port
- Max APDU length
- Segmentation support
- Vendor ID
- Register status
- Register record ID

- Last matched time

11.10 Live Points Tab

The Live Points tab shall show a table of live BACnet objects.

Required columns:

- Object type
- Object name
- Object instance
- Present value
- Units
- Status flags
- Reliability

The tool shall support common BACnet object types including:

- Analog Input
- Analog Output
- Analog Value
- Binary Input
- Binary Output
- Binary Value
- Multi-state Input
- Multi-state Output
- Multi-state Value
- Schedule
- Trend Log
- Device

11.11 Object Tree Tab

The Object Tree tab shall present objects grouped by object type.

Example groupings:

- Analog Inputs
- Analog Outputs
- Binary Inputs
- Binary Outputs
- Multi-state Inputs
- Multi-state Outputs
- Schedules
- Trend Logs

The user should be able to expand and collapse object groups.

11.12 BACnet Object Reading Behaviour

For each BACnet object, the tool should attempt to read:

- objectIdentifier
- objectName
- objectType
- presentValue
- units where applicable
- statusFlags
- reliability
- description where available
- outOfService
- priorityArray where applicable
- relinquishDefault where applicable

11.13 How it Should Behave

The user shall be able to:

- Configure BACnet discovery settings.
- Start BACnet discovery.
- Import a BACnet Device Register.
- View discovered devices.
- Click a device.
- View live BACnet objects.
- Read or refresh object properties.
- Compare devices and objects against the register.
- Export object lists.
- View BACnet issues.

11.14 Finished Result

The finished page shall look like the BACnet Discovery page in the React mockup:

- Discovery controls at top.
- Summary metrics.
- BACnet device table on the left.
- Device inspector on the right.
- Live point data table.
- Object tree view.
- Information icons.

11.15 Success Checks

Success shall be checked by confirming that:

- BACnet devices can be discovered using Who-Is/I-Am.
- Discovered devices populate the table.
- Imported BACnet register records can be compared.
- Clicking a BACnet device opens live point information.
- BACnet object values and status data are visible.

- Object lists can be exported.
- Missing, unmatched and unexpected BACnet devices are flagged.

---

12. MQTT Discovery Page Specification

12.1 What Needs to be Built

Build an MQTT Discovery page that allows users to connect to an MQTT broker, subscribe to topics, inspect live payloads, extract assets and points, compare topics against an MQTT Device Register, and validate payload structure.

The page shall include:

- Broker status/control area.
- Topic subscription controls.
- MQTT register import.
- Summary metric cards.
- Discovered topic/assets table.
- Clickable topic rows.
- Topic/asset inspector.
- Live JSON payload viewer.
- Extracted live points table.
- Register comparison panel.

12.2 Why it is Needed

MQTT discovery is required to confirm that the Smart integration broker is receiving data from expected assets, that topics are subscribable, and that payloads are correctly structured for downstream Smart Building use.

It is also required to confirm:

- Broker connectivity.
- TLS/certificate authentication.
- Expected topics are present.
- Payloads are valid JSON.
- Payloads follow expected schema or UDMI-style structure.
- Required points are present.
- Values, units and timestamps are available.
- Unexpected topics or assets are identified.

12.3 Where it Applies

This page applies to MQTT-based Smart Building integrations, including:

- Gateway-published BMS data.
- IoT sensor data.
- Smart Building platform ingestion feeds.
- UDMI-aligned telemetry.
- Broker-to-platform data streams.

- Translated BACnet-to-MQTT point data.

12.4 Required Inputs

The page shall support:

- MQTT broker endpoint.
- Broker port.
- TLS/certificate settings from Configuration page.
- Root topic filter.
- QoS.
- Subscription mode.
- MQTT Device Register import.

12.5 MQTT Device Register Import

Minimum expected columns:

- Project/site
- System
- Asset ID
- Asset name
- Expected topic
- Payload type
- Expected schema version
- Expected points
- Expected units
- Expected reporting interval
- Source protocol, if translated
- Notes

12.6 Required Outputs

The page shall show:

- Expected assets.
- Subscribed assets.
- Topics discovered.
- Schema/payload issues.
- Discovered topic table.
- Live payload viewer.
- Extracted point list.
- Register comparison.

12.7 Discovered Topics & Assets Table

The table shall include:

- Topic
- Asset
- Payload type
- Message rate

- Payload status
- Register status
- Last seen timestamp

Rows shall be clickable.

Clicking a row shall update the topic/asset inspector.

12.8 Topic / Asset Inspector

The inspector shall show:

- Asset name
- Subscription status
- Register match status
- Live message rate
- Message count
- Overview tab
- Live Payload tab
- Points tab
- Metadata tab

Required actions:

- Pause stream
- Export payload
- Compare to register

12.9 Live Payload Tab

The Live Payload tab shall show:

- Topic overview.
- Payload metadata.
- Code-style JSON viewer.
- Live extracted points.
- Register comparison summary.

The JSON viewer should support:

- Monospace formatting.
- Syntax-like visual formatting.
- Scrolling.
- Copy action, if developed.
- Latest message display.

12.10 Extracted Points

The extracted points table shall include:

- Point name
- Value

- Units
- Timestamp

The extraction logic should parse live JSON payloads and identify point-level data.

For UDMI-style payloads, the tool should support fields such as:

- version
- timestamp
- sequence number
- system/state payload
- pointset payload
- metadata payload
- config payload where applicable
- point names
- point values
- units
- status or quality fields where available

12.11 Payload Status Logic

Suggested status values:

- Valid
- Warning
- Invalid
- Missing expected field
- Invalid JSON
- Schema mismatch
- Stale payload
- Unmatched
- Partial

12.12 How it Should Behave

The user shall be able to:

- Confirm broker connection.
- Enter or confirm a root topic filter.
- Select QoS.
- Start subscription.
- View discovered topics.
- Click a topic or asset.
- View live payloads.
- Pause and resume live stream.
- View extracted points.
- Compare against MQTT register.
- Export payloads.

12.13 Finished Result

The finished page shall look like the MQTT Discovery page in the React mockup:

- Broker status and subscription controls.
- Summary metrics.
- Topic table on the left.
- Detailed live payload inspector on the right.
- JSON viewer.
- Extracted live points table.
- Register comparison panel.
- Clickable information icons.

12.14 Success Checks

Success shall be checked by confirming that:

- Broker connection status is shown.
- The tool can subscribe to a configured topic filter.
- Topics appear in the table.
- Payloads can be viewed live.
- Valid JSON is recognised.
- Invalid payloads are flagged.
- Extracted points are displayed.
- Register comparison results are calculated.
- Clicking a row updates the inspector.

---

13. Data Validation Page Specification

13.1 What Needs to be Built

Build a Data Validation page that validates expected BACnet and MQTT data against imported validation files and confirms the accuracy of BACnet-to-MQTT mappings.

The page shall include:

- Validation mode selector.
- Validation file import.
- Run validation action.
- Summary metric cards.
- Asset validation results table.
- Clickable asset rows.
- Asset validation inspector.
- BACnet point validation results.
- MQTT payload validation results.
- BACnet-to-MQTT comparison table.
- Point detail drilldown.
- Issue summary.
- Report and export actions.

13.2 Why it is Needed

Discovery confirms that devices and topics are visible. Validation confirms that the data is complete, correct, usable and aligned with expected Smart Building integration requirements.

The validation page is needed to prove:

- Expected assets are online.
- Expected BACnet points are present.
- Expected MQTT payloads are present.
- Payloads are correctly structured.
- Expected point names exist.
- Live values are present.
- Units are correct.
- Data is not stale.
- BACnet values are correctly translated to MQTT values.
- Mismatches are visible and reportable.

13.3 Where it Applies

This page applies to:

- BACnet point validation.
- MQTT payload validation.
- MQTT point extraction validation.
- BACnet-to-MQTT mapping validation.
- Smart Building data readiness checks.
- Commissioning evidence packs.

13.4 Required Validation Modes

The page shall include the following modes:

1. Validation Overview
2. MQTT Payload Validation
3. BACnet Point Validation
4. BACnet ↔ MQTT Comparison

13.5 Required Inputs

The page shall support importing validation files.

The imported files may include:

- Asset validation register.
- BACnet point register.
- MQTT point/payload register.
- BACnet-to-MQTT mapping file.
- Expected units file.
- Tolerance file.
- Project/system filter file where applicable.

13.6 Minimum Validation File Fields

Asset-Level Fields

- Project/site
- System
- Asset ID
- Asset name
- Source protocol
- Expected online status
- Expected topic or device reference
- Location

BACnet Point Fields

- Asset ID
- Device instance
- BACnet network
- Object type
- Object instance
- Object name
- Expected point name
- Expected units
- Expected value type
- Required/optional flag

MQTT Point Fields

- Asset ID
- Topic
- Payload type
- JSON path or field name
- Expected point name
- Expected units
- Expected value type
- Required/optional flag
- Expected reporting interval

Mapping Fields

- Asset ID
- BACnet device instance
- BACnet object type
- BACnet object instance
- BACnet object name
- BACnet units
- MQTT topic
- MQTT field/path
- MQTT units
- Tolerance
- Mapping required flag

13.7 Required Outputs

The page shall show:

- Expected assets.
- Online and valid assets.
- Point issues.
- Translation mismatches.
- Validation results table.
- Selected asset detail panel.
- Issue summary.
- Point detail drilldown.
- Exportable issue register.
- Exportable validation report.

13.8 Validation Results Table

The table shall include:

- Asset
- Validation source
- Points valid
- Result
- Issues
- Last run
- Project

Rows shall be clickable.

Clicking a row shall update the asset validation inspector.

13.9 Asset Validation Inspector

The inspector shall show:

- Selected asset name.
- Asset type badge.
- Online status.
- Source protocol badge.
- Generate report action.
- Export issues action.
- Open point detail action.

Required tabs:

- Overview
- Point Issues
- BACnet ↔ MQTT Compare

13.10 BACnet ↔ MQTT Compare Tab

The compare tab shall include a mapping comparison table.

Required columns:

- BACnet object name
- BACnet live value
- MQTT field
- MQTT live value
- Unit
- Alignment result
- Tolerance

Suggested alignment statuses:

- Pass
- Warning
- Fail

Comparison logic shall consider:

- Whether BACnet source point exists.
- Whether MQTT mapped field exists.
- Whether both values are present.
- Whether values are numerically aligned within tolerance.
- Whether boolean/binary values match.
- Whether enumerated values match expected mapping.
- Whether units match.
- Whether timestamps are current enough.
- Whether values are stale.

13.11 Point Detail Drilldown

When a mapping or point row is selected, the user shall be able to view point detail.

Required point detail sections:

- BACnet details
- MQTT details
- Comparison details
- Available actions

BACnet Details

Required fields:

- Object type
- Object instance
- Object name
- Present value
- Units
- Status flags
- Reliability
- Timestamp

MQTT Details

Required fields:

- Topic
- Field/path
- Value
- Units
- Schema type
- Quality
- Timestamp

Comparison Details

Required fields:

- Difference
- Tolerance
- Result
- Reason

13.12 Issue Summary

The issue summary panel shall show grouped issue types.

Minimum issue categories:

- Missing points
- Null or stale values
- Schema/type errors
- Unit mismatches
- Out-of-tolerance values
- Unexpected points
- Register mismatch

13.13 How it Should Behave

The user shall be able to:

- Import validation files.
- Select validation mode.
- Run validation.
- Filter by environment and project.
- View high-level metrics.
- Click an asset.
- View point-level results.
- View BACnet-to-MQTT mapping results.
- Drill into a failing point.
- Understand the reason for a failure.
- Export issues.
- Generate a report.

13.14 Finished Result

The finished page shall look like the Data Validation page in the React mockup:

- Mode selector at the top.
- Summary metric cards.
- Validation results table on the left.
- Detailed inspector on the right.
- Mapping comparison table.
- Issue summary.
- Point detail drilldown.
- Information icons.

13.15 Success Checks

Success shall be checked by confirming that:

- Validation files can be imported.
- Validation can be run.
- Assets are listed with pass/warning/fail results.
- Clicking an asset updates the inspector.
- BACnet point validation results are visible.
- MQTT payload validation results are visible.
- BACnet-to-MQTT mapped values can be compared.
- Tolerance failures are correctly flagged.
- Missing and stale values are identified.
- Reports and issue exports can be generated.

---

14. Reports Page Specification

14.1 What Needs to be Built

Build a Reports page that allows users to generate and export commissioning evidence from discovery and validation workflows.

14.2 Why it is Needed

The commissioning process requires clear evidence that devices, points, topics and translated data have been tested and validated.

The Reports page provides the final output required for project records, client review, issue management and handover.

14.3 Required Report Types

The application shall support the following report outputs:

- IP discovery report.
- BACnet discovery report.

- MQTT discovery report.
- Data validation summary report.
- BACnet point issue register.
- MQTT payload issue register.
- BACnet-to-MQTT mapping comparison report.
- Full commissioning evidence pack.

14.4 Required Export Formats

The tool should support:

- PDF for formal reports.
- CSV for raw data export.
- XLSX for issue registers and validation outputs.
- JSON where useful for raw payload or API-based export.

14.5 Report Content

Reports should include:

- Project name.
- Site name.
- Date/time generated.
- Tool version.
- User who generated the report.
- Configuration summary.
- Imported register reference.
- Discovery summary.
- Validation summary.
- Pass/warning/fail metrics.
- Detailed issue list.
- Evidence tables.
- Notes added by commissioning engineer.

14.6 Success Checks

Success shall be checked by confirming that:

- Reports can be generated from completed discovery/validation results.
- Reports include summary and detailed evidence.
- Reports can be downloaded.
- Exported files match the filtered view where filters are applied.

---

15. Data Model Requirements

15.1 Core Entities

The backend should support the following core entities:

- Project

- Site
- System
- Device
- Asset
- IP Scan Result
- BACnet Device
- BACnet Object
- MQTT Topic
- MQTT Payload
- Extracted MQTT Point
- Validation Run
- Validation Result
- Mapping Result
- Issue
- Report
- User Note

15.2 Device Entity

Recommended fields:

- device_id
- project_id
- site_id
- system_id
- device_name
- asset_id
- hostname
- ip_address
- subnet
- vlan_id
- mac_address
- device_type
- manufacturer
- model
- location
- expected_ports
- observed_ports
- register_status
- online_status
- last_seen
- notes

15.3 BACnet Device Entity

Recommended fields:

- bacnet_device_id
- project_id
- device_name
- device_instance
- bacnet_network

- ip_address
- udp_port
- vendor_id
- vendor_name
- model_name
- firmware_revision
- max_apdu_length
- segmentation_supported
- object_count
- online_status
- register_status
- last_discovered

15.4 BACnet Object Entity

Recommended fields:

- bacnet_object_id
- bacnet_device_id
- object_type
- object_instance
- object_name
- description
- present_value
- units
- status_flags
- reliability
- out_of_service
- priority_array
- relinquish_default
- last_read

15.5 MQTT Topic Entity

Recommended fields:

- mqtt_topic_id
- project_id
- asset_id
- topic
- payload_type
- schema_version
- qos
- retained
- message_rate
- message_count
- last_seen
- payload_status
- register_status

15.6 MQTT Extracted Point Entity

Recommended fields:

- mqtt_point_id
- mqtt_topic_id
- asset_id
- point_name
- json_path
- value
- units
- quality
- timestamp
- schema_type
- last_seen

15.7 Mapping Result Entity

Recommended fields:

- mapping_result_id
- validation_run_id
- asset_id
- bacnet_device_instance
- bacnet_object_type
- bacnet_object_instance
- bacnet_object_name
- bacnet_live_value
- bacnet_units
- mqtt_topic
- mqtt_field
- mqtt_live_value
- mqtt_units
- tolerance
- result
- difference
- reason
- timestamp

15.8 Issue Entity

Recommended fields:

- issue_id
- validation_run_id
- project_id
- asset_id
- device_id
- point_name
- issue_type
- severity

- description
- source
- suggested_action
- status
- created_at
- resolved_at
- notes

---

16. Import File Behaviour

16.1 General Import Requirements

The application shall support importing register and validation files.

For each import, the tool shall:

- Validate file type.
- Validate required columns.
- Show import success/failure.
- Show row count.
- Show rejected rows.
- Allow the user to download an import error report.
- Store import metadata.

16.2 Supported File Types

Preferred formats:

- XLSX
- CSV

Optional future formats:

- JSON
- XML

16.3 Import Error Handling

The tool shall identify:

- Missing required columns.
- Invalid IP addresses.
- Invalid BACnet instance numbers.
- Duplicate device records.
- Duplicate point mappings.
- Invalid MQTT topics.
- Invalid units.
- Empty required fields.

---

17. Validation Logic Requirements

17.1 Asset-Level Validation

For each expected asset, the tool shall determine:

- Is the asset expected?
- Was the asset discovered?
- Is the asset online?
- Does the asset match the imported register?
- Are all required data points present?
- Are values being received?
- Are values current?
- Are values in the expected format?

17.2 BACnet Point Validation

For each expected BACnet point, validate:

- Device exists.
- Object exists.
- Object type matches.
- Object instance matches.
- Object name matches, where required.
- Present value is readable.
- Value is not null.
- Units match expected units.
- Status flags indicate normal operation or are clearly flagged.
- Reliability does not indicate fault, unless expected.

17.3 MQTT Payload Validation

For each expected MQTT topic/payload, validate:

- Topic exists.
- Payload received within expected time interval.
- Payload is valid JSON.
- Payload follows expected structure.
- Required fields are present.
- Required points are present.
- Values are present.
- Units are present where required.
- Timestamps are present and current.
- Schema version matches expected version where required.

17.4 BACnet-to-MQTT Mapping Validation

For each mapping, validate:

- BACnet source point exists.
- BACnet source value is readable.

- MQTT mapped field exists.
- MQTT mapped value is present.
- Units match or are correctly converted.
- Value is aligned within tolerance.
- Boolean values match.
- Enumerated values are correctly mapped.
- Timestamp difference is within acceptable range.

17.5 Result Statuses

Use the following result statuses:

- Pass
- Warning
- Fail
- Not Tested
- Not Applicable

17.6 Severity Levels

Use the following issue severity levels:

- Critical
- High
- Medium
- Low
- Informational

---

18. Live Data Requirements

18.1 IP Live Data

The tool should be able to show latest available network observations such as:

- Ping response time.
- Packet loss.
- Last seen timestamp.
- Link status where available.
- MAC address where available.
- Open services.

18.2 BACnet Live Data

The tool should be able to show:

- Live present values.
- Units.
- Status flags.
- Reliability.
- Last read timestamp.

- Object properties.

18.3 MQTT Live Data

The tool should be able to show:

- Latest payload.
- Message rate.
- Last message timestamp.
- Message count.
- Extracted point values.
- Payload validation status.

---

19. Security Requirements

The tool shall be designed with appropriate security controls.

Minimum requirements:

- Support MQTT over TLS.
- Support certificate-based MQTT authentication.
- Store private keys securely.
- Do not expose certificate private keys in the UI.
- Mask sensitive fields such as passwords.
- Support role-based access where user management is introduced.
- Log user actions relevant to configuration and reporting.
- Avoid storing unnecessary credentials.
- Protect imported data and reports.

---

20. Audit and Logging Requirements

The application should log:

- Configuration changes.
- Register imports.
- Discovery runs.
- Validation runs.
- Report generation.
- Export actions.
- Errors and failed protocol connections.
- User notes and issue status changes.

Logs should include:

- Timestamp.
- User, where available.
- Action type.
- Project/site context.

- Result.
- Error message where applicable.

---

21. Performance Requirements

The tool should be able to handle realistic project datasets.

Suggested minimum targets:

- IP scan results: at least 5,000 devices.
- BACnet devices: at least 1,000 devices.
- BACnet objects: at least 250,000 objects.
- MQTT topics: at least 100,000 topics.
- Validation mappings: at least 250,000 point mappings.

The UI should remain responsive when large datasets are loaded.

Recommended UI features:

- Pagination.
- Search.
- Filtering.
- Lazy loading.
- Virtualised tables for large datasets.
- Background job processing for long scans.

---

22. Error Handling Requirements

The tool shall clearly show errors to the user.

Examples:

- Broker connection failed.
- BACnet discovery timeout.
- IP scan failed.
- Register import failed.
- Required columns missing.
- Payload is invalid JSON.
- BACnet object read failed.
- Certificate expired.
- NTP not synchronised.

Error messages should be specific, actionable and linked to the affected device, topic, point or configuration field.

---

23. Accessibility and Usability Requirements

The application should:

- Use readable font sizes.
- Use sufficient colour contrast.
- Not rely on colour alone for status.
- Include text labels with icons.
- Provide clear focus states.
- Support keyboard navigation where possible.
- Provide meaningful tooltips/popovers.
- Keep tables sortable and searchable.

---

24. Acceptance Criteria Summary

The finished application shall be considered successful when the following criteria are met:

Configuration

- All required configuration sections are present.
- Settings can be viewed and edited.
- Invalid settings are flagged.
- Status indicators work.
- Information icons are clickable.

IP Scanner

- IP scans can be configured and run.
- Results are displayed in a table.
- Results compare against an imported register.
- Rogue and missing devices are flagged.
- Clicking a device opens a detail inspector.
- Results can be exported.

BACnet Discovery

- BACnet devices can be discovered.
- BACnet devices compare against an imported register.
- Clicking a device shows live object data.
- BACnet objects show present value, units, status flags and reliability.
- Object lists can be exported.

MQTT Discovery

- Broker connection status is displayed.
- Topics can be subscribed to.
- Payloads can be viewed live.
- Points can be extracted from payloads.
- Payloads are validated.
- Topics/assets compare against an imported register.

Data Validation

- Validation files can be imported.
- Validation can be run.
- Assets show pass/warning/fail status.
- Device and point issues are visible.
- BACnet-to-MQTT mappings can be compared.
- Out-of-tolerance values are flagged.
- Point-level detail is available.

Reports

- Reports can be generated.
- Issue registers can be exported.
- Evidence packs include summary and detailed results.

---

25. Definition of Done

The development work shall be considered complete when:

- The React UI reflects the approved mockup.
- All pages are navigable.
- Information icons open explanatory popovers.
- Clickable device/topic/asset rows update inspector panels.
- Import workflows support required register types.
- Discovery workflows produce structured result data.
- Validation workflows calculate pass/warning/fail results.
- BACnet-to-MQTT comparison logic is implemented.
- Reports and exports can be generated.
- Errors are clearly shown.
- Core workflows have been tested with representative project data.
- The tool can be demonstrated end-to-end using sample IP, BACnet, MQTT and validation datasets.

---

26. Suggested Development Approach

A suitable delivery approach would be:

1. Build the application shell and navigation.
2. Build Configuration page and persistent settings model.
3. Build import framework for registers.
4. Build IP Scanner UI and mocked scan result workflow.
5. Connect IP Scanner to backend scan service.
6. Build BACnet Discovery UI and mocked BACnet result workflow.
7. Connect BACnet Discovery to BACnet service/library.
8. Build MQTT Discovery UI and mocked broker/topic workflow.
9. Connect MQTT Discovery to broker subscription service.
10. Build Data Validation UI and validation result model.

11. Implement validation logic.
12. Implement BACnet-to-MQTT mapping comparison.
13. Build Reports page and export engine.
14. Add audit logging and error handling.
15. Test with sample project datasets.
16. Refine performance for large datasets.

---

27. Key Developer Notes

- The UI should not be treated as a static dashboard. Table rows, tabs, information icons and inspector panels must be interactive.
- Discovery and validation operations may need to run as asynchronous background jobs.
- The frontend should be designed to poll job status or receive updates via WebSocket/server-sent events where live updates are required.
- The data model should preserve raw discovered data as well as interpreted/validated data.
- The system should distinguish between expected, discovered, matched, partial, unmatched and unexpected records.
- All validation failures should be traceable back to source data.
- Reports should be reproducible from stored validation run records.
- The application should be structured so additional protocols can be added later, such as Modbus TCP, SNMP, OPC UA or REST API validation.

---

28. Future Expansion Considerations

The following features are not required for the first build but should be considered in the architecture:

- Modbus TCP discovery and validation.
- SNMP discovery and validation.
- OPC UA discovery and validation.
- REST API endpoint validation.
- User authentication and role-based access.
- Multi-project dashboard.
- Issue assignment and workflow status.
- Integration with Jira, Azure DevOps or other issue trackers.
- Automated scheduled validation runs.
- Notification rules.
- Comparison between historical validation runs.
- Commissioning sign-off workflow.
- QR code linkage to asset records.
- Direct export to Smart Building platform asset/point libraries.